

Day 3 — Production-Style Python

Writing Code Like a Real Project, Not a Script — Notes for Recap

1. Functions with Clear Inputs/Outputs

What does "clear I/O" mean?

A well-designed function should have a clear, predictable input (what it takes in) and output (what it returns) — instead of relying on variables floating around outside the function, or printing results instead of returning them.

```
# Unclear - relies on a global variable, prints instead of returning
total = 0
def add_to_total(n):
    global total
    total += n
    print(total)

# Clear - explicit input, explicit output, no hidden state
def add(a, b):
    return a + b

result = add(5, 3)    # result = 8
```

Key takeaway: A function with clear inputs and outputs is easy to test in isolation and easy to reuse elsewhere in a pipeline — you always know exactly what goes in and what comes out.

2. Type Hints

Type hints let you document what type of data a function expects and returns, without changing how Python actually runs the code (they are not strictly enforced, but tools and editors use them to catch mistakes early).

```
def calculate_total(price: float, quantity: int) -> float:
    return price * quantity

# Without a type hint, it is not obvious what types are expected:
def calculate_total_unclear(price, quantity):
    return price * quantity
```

Key takeaway: Type hints make code self-documenting. In a data pipeline, they help you instantly see whether a function expects a string, a number, or a list — without reading the entire function body.

3. pathlib

`pathlib` is Python's modern, object-oriented way of working with file paths — a cleaner replacement for manually joining strings with slashes.

```
from pathlib import Path

data_folder = Path("data")
file_path = data_folder / "sales.csv"    # builds the path safely

print(file_path)                        # data/sales.csv
print(file_path.exists())               # True or False
print(file_path.suffix)                 # .csv
```

Key takeaway: *pathlib works correctly across Windows, Mac, and Linux automatically — unlike manually typing "data/" + filename, which can break depending on the operating system.*

4. logging

The logging module is the proper way to record what a program is doing — instead of scattering `print()` statements everywhere, which have to be manually removed later and give no context (timestamps, severity level).

```
import logging

logging.basicConfig(level=logging.INFO)

logging.info("Pipeline started")
logging.warning("Missing value found in row 42")
logging.error("Failed to connect to database")
```

Key takeaway: *Logging gives you a permanent, timestamped record of what happened during a pipeline run — essential for debugging a job that failed overnight when nobody was watching.*

5. Config Files

A config file stores settings (file paths, API keys, batch sizes) separately from the code, so they can be changed without touching the actual script.

```
# config.py
DATA_FOLDER = "data/raw"
BATCH_SIZE = 500
API_URL = "https://api.example.com/orders"

# main.py
from config import DATA_FOLDER, BATCH_SIZE
print(f"Reading from {DATA_FOLDER} in batches of {BATCH_SIZE}")
```

Key takeaway: *Hardcoding values directly in your logic makes a script fragile. Config files let you change behavior (e.g., point to a different folder) without editing the pipeline code itself.*

6. Environment Variables

Environment variables store sensitive or environment-specific values (passwords, API keys, database URLs) outside the codebase entirely — usually in a `.env` file that is never committed to Git.

```
# .env file (never pushed to GitHub)
DB_PASSWORD=supersecret123

# in Python, using python-dotenv
import os
from dotenv import load_dotenv

load_dotenv()
password = os.getenv("DB_PASSWORD")
```

Key takeaway: *Never hardcode passwords or API keys directly in your code. Environment variables keep secrets out of your GitHub repo — a real security requirement, not just a best practice.*

7. Modular Project Structure

Instead of writing one long script that does everything top to bottom, a modular project splits logic into separate files/functions with a clear responsibility each.

```
my_project/
  main.py          # runs the pipeline
  extract.py       # functions to pull data
  transform.py     # functions to clean/transform data
  load.py          # functions to save/load data
  config.py        # settings
  .env             # secrets (not committed to Git)
```

Key takeaway: *A modular structure means you can test, debug, and reuse each part independently — one messy 300-line script is much harder to fix than five small, focused files.*

8. venv/Poetry + Dependency Pinning

A virtual environment isolates the packages installed for one project from every other project on your machine, avoiding version conflicts.

```
# creating and activating a virtual environment
python -m venv venv
venv\Scripts\activate      # Windows
source venv/bin/activate   # Mac/Linux

# saving exact package versions used
pip freeze > requirements.txt

# installing the same versions elsewhere
pip install -r requirements.txt
```

Dependency pinning means recording the exact version of each package used (e.g. `pandas==2.1.0`) so the project behaves identically on any machine — including a teammate's laptop or a production server.

Key takeaway: Avoid writing one giant script with no structure. A virtual environment plus a requirements.txt file means your project can be reproduced exactly, anywhere, without "it works on my machine" problems.

Day 3 Summary — Production-Style Python

Concept	One-Line Recap
Clear I/O Functions	Explicit inputs and outputs instead of hidden global state
Type Hints	Self-documenting code that shows expected data types
pathlib	Safe, cross-platform way to build and work with file paths
logging	Timestamped, permanent records of what a program did - better than print()
Config Files	Settings kept separate from logic, so behavior can change without code edits
Environment Variables	Secrets (passwords, API keys) kept out of the codebase entirely
Modular Structure	Splitting one script into focused, independently testable files
venv/Poetry	Isolated, reproducible environments with exact package versions pinned

Overall takeaway: Day 3 is about the shift from "a script that works on my machine" to "a project that works anywhere." Clear functions, logging, config separation, and environment isolation are what separate a hobby script from something that could actually run in production.